RELATIONAL DATA MODEL

Basic Concepts, Integrity Constraints, and Update Operations

Outline

1. Relational Model Concepts
2. Characteristics of Relations
3. Relational Constraints
4. Update Operations on Relations

1. Relational Model Concepts

- The relational Model of Data is based on the notion of a **Relation.**

- A Relation is a **set**.

- The strength of the relational approach to data management comes from the *formal foundation provided by the theory of relations*.

The model was first proposed by E.F. Codd of IBM in 1970 in the following paper:
"A Relational Model for Large Shared Data Banks,"
Communications of the ACM, June 1970.

1.1. Informal definitions (1)

- **RELATION:** A table of values
- A relation may be thought of as a **set of rows.**
- Each row of the relation may be given an identifier.
- Each column typically is called by its column name (or attribute name).

1.1. Informal definitions (2)

An example of a database that stores student records and their grades.

STUDENT	Name	StudentNumber	Class	Major
	Smith	17	1	CS
	Brown	8	2	CS

COURSE	CourseName	CourseNumber	CreditHours	Department
	Intro to Computer Science	CS1310	4	CS
	Data Structures	CS3320	4	CS
	Discrete Mathematics	MATH2410	3	MATH
	Database	CS3380	3	CS

SECTION	SectionIdentifier	CourseNumber	Semester	Year	Instructor
	85	MATH2410	Fall	98	King
	92	CS1310	Fall	98	Anderson
	102	CS3320	Spring	99	Knuth
	112	MATH2410	Fall	99	Chang
	119	CS1310	Fall	99	Anderson
	135	CS3380	Fall	99	Stone

GRADE_REPORT	StudentNumber	SectionIdentifier	Grade
	17	112	B
	17	119	C
	8	85	A
	8	92	A
	8	102	B
	8	135	A

PREREQUISITE	CourseNumber	PrerequisiteNumber
	CS3380	CS3320
	CS3380	MATH2410
	CS3320	CS1310

1.2. Formal definitions (1)

- A Relation may be defined in multiple ways.
- A **domain** is a set of atomic values.
- Each attribute **A** has a **domain** denoted **dom(A)**.
- The **schema** of a relation R is a list of **distinct attributes (A1, A2, …, An)**.
- **R(A1, A2, …, An)** denotes a relation **R** with schema **(A1, A2, …, An)**.
- The **degree** of a relation is the number of attributes in its schema.

Example:

CUSTOMER(Cust-id, Cust-name, Address, Phone#)

Here, CUSTOMER is a relation of degree four defined over the attributes Cust-id, Cust-name, Address, Phone#, each of which has a domain For example, the domain of Cust-id is the set of 10 digit numbers.

1.2. Formal definitions (2)

- A **tuple t over a schema** $(A_1, A_2, ...A_n)$ is an ordered list of values $(v_1, v_2, ...v_n)$ such that $v_i \in$ dom(A_i) or v_i = null.
- A **relation instance r** (or **relation state**) of a relation $R(A_1, A_2, ...A_n)$ is a set of tuples over $(A_1, A_2, ...A_n)$.

Example:

Each row in the CUSTOMER table is a tuple and consists of four values.

<632895, "John Smith", "101 Main St. Atlanta, GA 30332", "(404) 894-2000"> is a tuple belonging to the CUSTOMER relation.

- The terms **intension** for a *relation schema* and **extension** for a *relation instance* are also used.

1.2. Formal definitions (3)

- Consider a relation $R(A_1, A_2, \ldots, A_n)$

  A relation instance r(R) is **a subset of the Cartesian product** $dom(A_1) \times dom(A_2) \times \ldots \times dom(A_n)$:

  $r(R) \subseteq dom(A_1) \times dom(A_2) \times \ldots \times dom(A_n)$

- The cardinality of the Cartesian product is
  $|dom(A_1)| \times |dom(A_2)| \times \ldots \times |dom(A_n)|$
  where $|D|$ is the cardinality of D.

Example:

Relation $R(A_1, A_2)$
$dom(A_1) = \{0,1\}$ $dom(A_2) = \{a,b,c\}$

$dom(A_1) \times dom(A_2) = \{<0,a>, <0,b>, <0,c>, <1,a>, <1,b>, <1,c>\}$

$r(R) = \{<0,a>, <0,b>, <1,c>\}$

1.2. Formal definitions (4)

The attributes and tuples of a relation STUDENT.

Relation name

Attributes

Tuples

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

1.2. Formal definitions (5)

Definition summary

Informal Terms	Formal Terms
Table	Relation
Column header	Attribute
Row	Tuple
Table Definition	Schema of Relation
Populated Table	Instance, State, Extension

Note:

Whereas languages like SQL use the informal terms of TABLE (e.g. CREATE TABLE), COLUMN (e.g. SYSCOLUMN variable) etc., the relational database textbooks present the model and operations on it using the formal terms.

2. Characteristics of relations (1)

- **Ordering of tuples in a relation r(R)**:
  The tuples are *not* considered to be ordered, even though they appear to be so in the tabular form.

Example:

The tables below represent of the same relation.

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25

STUDENT	Name	SSN	HomePhone	Address	OfficePhone	Age	GPA
	Dick Davidson	422-11-2320	null	3452 Elgin Road	749-1253	25	3.53
	Barbara Benson	533-69-1238	839-8461	7384 Fontana Lane	null	19	3.25
	Charles Cooper	489-22-1100	376-9821	265 Lark Lane	749-6492	28	3.93
	Katherine Ashly	381-62-1245	375-4409	125 Kirby Road	null	18	2.89
	Benjamin Bayer	305-61-2435	373-1616	2918 Bluebonnet Lane	null	19	3.21

2. Characteristics of relations (2)

- **Ordering of attributes in a relation schema R** (and of values within each tuple):
  We will consider the attributes in $R(A_1, A_2, ..., A_n)$ and the values in $t = < v_1, v_2, ..., v_n >$ to be *ordered*.

- However, a more general *alternative definition* of relation does not require this ordering.

- Example:
  A tuple can be considered as a set of (<attribute>, <value>) pairs. Then, the ordering of attributes in a tuple is not important because each value appears with the name of the corresponding attribute.

t = < (Name, Dick Davidson),(SSN, 422-11-2320),(HomePhone, null),(Address, 3452 Elgin Road), (OfficePhone, 749-1253),(Age, 25),(GPA, 3.53)>

t = < (Address, 3452 Elgin Road),(Name, Dick Davidson),(SSN, 422-11-2320),(Age, 25), (OfficePhone,749-1253),(GPA, 3.53),(HomePhone, null)>

2. Characteristics of relations (3)

- **Values in a tuple**: All values are considered *atomic* (indivisible).

- This assumption is called **First Normal Form** Assumption.

  Relaxing this assumption leads to **non-first normal form** or **nested** relations.

- A special **null** value is used to represent values that are unknown or inapplicable to certain tuples.

2. Characteristics of relations (4)

Notation

- **Component values** of a tuple t:
  $t[A_i]$ and $t.A_i$ refer to the value of attribute A_i for tuple t.
  $t[A_u, A_v, ..., A_w]$ and $t.(A_u, A_v, ..., A_w)$ refer to the *subtuple of t* containing the values of attributes A_u, A_v, ..., A_w, respectively.

- **In general:**
  Q, R, S denote relation names.
  q, r, s denote relation instances.
  t, u, v denote tuples.

  The name of a relation schema (e.g. STUDENTS) *is also used for the instance of the relation*.

  An attribute can be qualified with the relation name to which it belongs using *dot notation*.
  E.g. STUDENT**.**Name, EMPLOYEE**.**Age

3. Relational constraints

- **Relational constraints** are *restrictions* that must hold on *all* valid relation instances.

- There are four main types of constraints:

  Domain constraints,
  Key constraints,
  Entity integrity constraints, and
  referential integrity constraints

3.1. Domain constraints

- A **domain constraint** specifies that the value of an attribute A must be from the domain dom(A).

- **Domains** can be specified using a **data type** (e.g. integer, real, fixed-length strings, variable length strings etc.) and possibly a **restriction**.

  Example: The domain of attribute Age of relation EMPLOYEE is the set of integers between 15 and 80.

  The domain of the attribute SSN is the set of valid 9-digit social security numbers.

- Domains can be given a **name**.

3.2. Key constraints (1)

- **Superkey of R:** A set of attributes SK of R such that no two tuples *in any valid relation instance r(R)* will have the same value for SK.

  That is, for any distinct tuples t_1 and t_2 in r(R), $t_1[SK] \neq t_2[SK]$.

- **Key of R:** A *"minimal" superkey*; that is, a superkey K such that the removal of any attribute from K results in a set of attributes that is not a superkey.

- A relation can have *more than one key*.

- Example: The CAR relation schema:
  CAR(State, Reg#, SerialNo, Make, Model, Year)
  has two keys: Key1 = {State, Reg#} and Key2 = {SerialNo}.

  Key1 and Key2 are also superkeys.

  {SerialNo, Make} is a superkey but *not* a key.

3.2. Key constraints (2)

- The value of a key can be used to *identify uniquely each tuple in the relation*.
- The set of attributes constituting a key is a *property of the relation schema*. It is a constraint that must hold *on every relation instance of the schema*.
- If a relation has *several* keys (**candidate keys**), one is chosen arbitrarily to be the **primary key**.
- Convention: The primary key attributes are *underlined* in the schema.
- Example:

CAR	LicenseNumber	EngineSerialNumber	Make	Model	Year
	Texas ABC-739	A69352	Ford	Mustang	96
	Florida TVP-347	B43696	Oldsmobile	Cutlass	99
	New York MPO-22	X83554	Oldsmobile	Delta	95
	California 432-TFY	C43742	Mercedes	190-D	93
	California RSK-629	Y82935	Toyota	Camry	98
	Texas RSK-629	U028365	Jaguar	XJS	98

3.3. Entity Integrity constraints

- **Entity Integrity**: The *primary key attributes* PK of each relation schema R cannot have null values in any tuple of r(R).

  This is because primary key values are used to *identify* the individual tuples.

- Note: Other attributes of R may be similarly constrained to disallow null values, *even though they are not members of the primary key*.

3.4. Relational Databases (1)

- The previous constraints are specified on *individual relations*.
  The fourth type of constraint is specified on two relations.

- To define this constraint we need the concept of Relational Database.

3.4. Relational Databases (2)

- **Relational Database Schema**: A set **S** = $\{R_1, R_2, ..., R_n\}$ of relation schemas and a set **IC** of constraints.

- **Relational Database State (**or **Instance) DB of S** is a set $\{r_1, r_2, ..., r_n\}$ of relation states such that:
  - r_i is a state of R_i and
  - the r_i's satify the integrity constraints specified in IC.

- **A Relational Database** is both a DB schema and an instance of this schema.

- Integrity constraints are *specified on a database schema* and are expected to hold *on every database state of that schema*.

3.4. Relational Databases (3)

Schema diagram for the COMPANY relational database schema; the primary keys are underlined.

EMPLOYEE

FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO

DEPARTMENT

DNAME	DNUMBER	MGRSSN	MGRSTARTDATE

DEPT_LOCATIONS

DNUMBER	DLOCATION

PROJECT

PNAME	PNUMBER	PLOCATION	DNUM

WORKS_ON

ESSN	PNO	HOURS

DEPENDENT

ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP

3.4. Relational Databases (4)

One possible relational database state corresponding to the COMPANY schema.

EMPLOYEE	FNAME	MINIT	LNAME	SSN	BDATE	ADDRESS	SEX	SALARY	SUPERSSN	DNO
	John	B	Smith	123456789	1965-01-09	731 Fondren, Houston, TX	M	30000	333445555	5
	Franklin	T	Wong	333445555	1955-12-08	638 Voss, Houston, TX	M	40000	888665555	5
	Alicia	J	Zelaya	999887777	1968-01-19	3321 Castle, Spring, TX	F	25000	987654321	4
	Jennifer	S	Wallace	987654321	1941-06-20	291 Berry, Bellaire, TX	F	43000	888665555	4
	Ramesh	K	Narayan	666884444	1962-09-15	975 Fire Oak, Humble, TX	M	38000	333445555	5
	Joyce	A	English	453453453	1972-07-31	5631 Rice, Houston, TX	F	25000	333445555	5
	Ahmad	V	Jabbar	987987987	1969-03-29	980 Dallas, Houston, TX	M	25000	987654321	4
	James	E	Borg	888665555	1937-11-10	450 Stone, Houston, TX	M	55000	null	1

DEPT_LOCATIONS	DNUMBER	DLOCATION
	1	Houston
	4	Stafford
	5	Bellaire
	5	Sugarland
	5	Houston

DEPARTMENT	DNAME	DNUMBER	MGRSSN	MGRSTARTDATE
	Research	5	333445555	1988-05-22
	Administration	4	987654321	1995-01-01
	Headquarters	1	888665555	1981-06-19

WORKS_ON	ESSN	PNO	HOURS
	123456789	1	32.5
	123456789	2	7.5
	666884444	3	40.0
	453453453	1	20.0
	453453453	2	20.0
	333445555	2	10.0
	333445555	3	10.0
	333445555	10	10.0
	333445555	20	10.0
	999887777	30	30.0
	999887777	10	10.0
	987987987	10	35.0
	987987987	30	5.0
	987654321	30	20.0
	987654321	20	15.0
	888665555	20	null

PROJECT	PNAME	PNUMBER	PLOCATION	DNUM
	ProductX	1	Bellaire	5
	ProductY	2	Sugarland	5
	ProductZ	3	Houston	5
	Computerization	10	Stafford	4
	Reorganization	20	Houston	1
	Newbenefits	30	Stafford	4

DEPENDENT	ESSN	DEPENDENT_NAME	SEX	BDATE	RELATIONSHIP
	333445555	Alice	F	1986-04-05	DAUGHTER
	333445555	Theodore	M	1983-10-25	SON
	333445555	Joy	F	1958-05-03	SPOUSE
	987654321	Abner	M	1942-02-28	SPOUSE
	123456789	Michael	M	1988-01-04	SON
	123456789	Alice	F	1988-12-30	DAUGHTER
	123456789	Elizabeth	F	1967-05-05	SPOUSE

3.5. Referential Integrity Constraints (1)

- A **Referential Integrity Constraint** involves two relations.
- It is used to specify a *relationship* among tuples in two relations: the **referencing relation** and the **referenced relation**.
- Informally this constraint states that *a tuple in one relation that refers to another relation must refer to an existing tuple in that relation*.

3.5. Referential Integrity Constraints (2)

- The referential integrity constraint is defined using the notion of the **foreign key of a relation**.

- Formally, a set of attributes **FK** in relation R_1 is a **foreign key** of R_1 that references the **primary key PK** of relation R_2 if it satisfies the following conditions:
  (1) The attribute(s) of FK have the same domain(s) as the attributes PK of R_2
  (2) For every tuple t_1 in R_1, there exists a tuple t_2 in R_2 such that $t_1[FK] = t_2[PK]$, or
  the value(s) in $t_1[FK]$ is (are) NULL.

- The attributes *FK are said to reference or refer to relation R_2*.

- If $t_1[FK] = t_2[PK]$ we say that *tuple t_1 references or refers to tuple t_2*.

3.5. Referential Integrity Constraints (3)

Example: Suppose that D is a foreign key of R_2 that refers to the primary key A of R_1.

R_1

A	B
a	b

R_2

C	D
c	a

This DB state satisfies the Referential Integrity Constraint.

R_1

A	B
a	b

R_2

C	D
c	a’

This DB state violates the Referential Integrity Constraint. It is not a legal (valid) state.

R_1

A	B
a	b

R_2

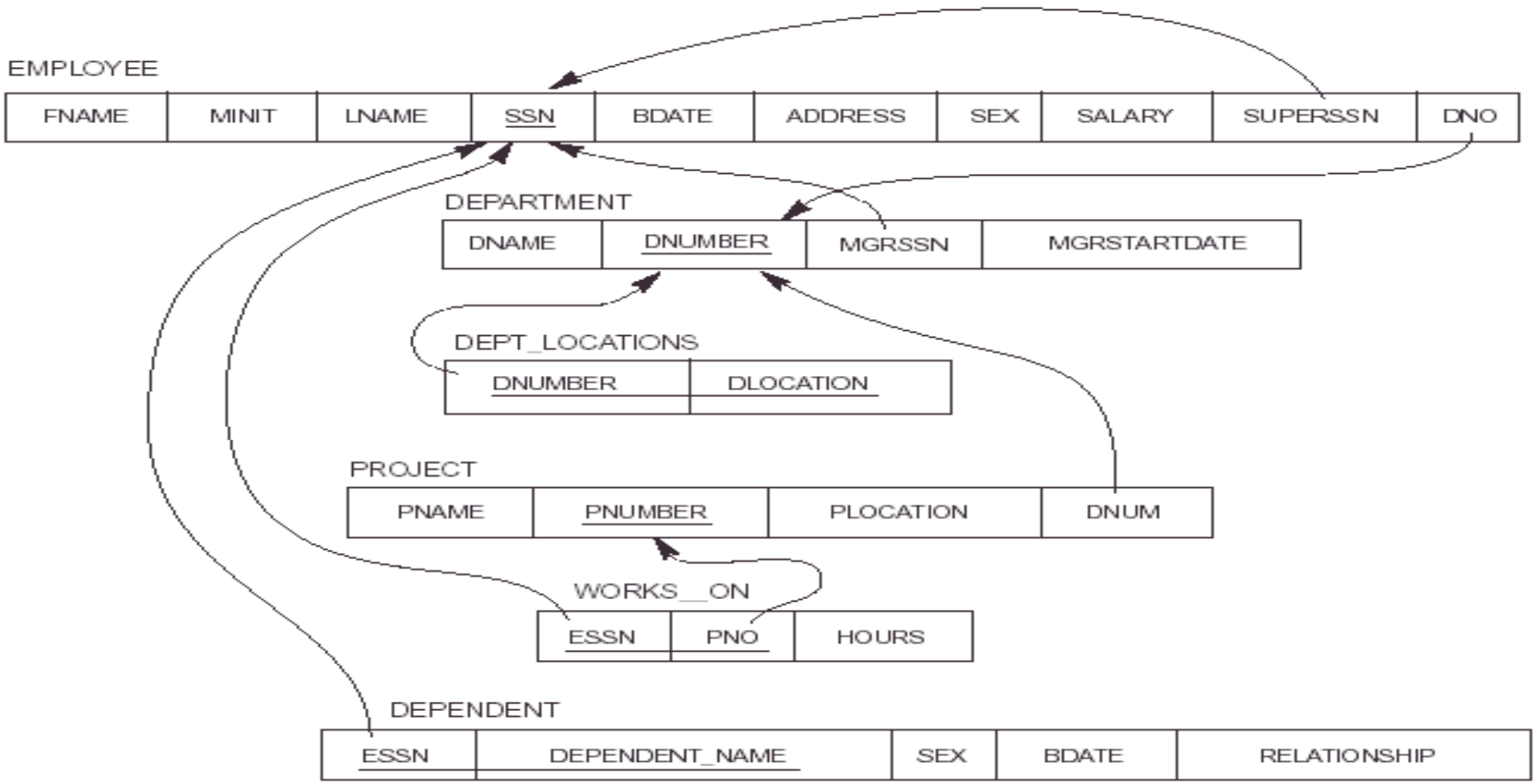
C	D
c	null

This DB state satisfies the Referential Integrity Constraint.

3.5. Referential Integrity constraints (4)

- A referential integrity constraint can be displayed in a relational database schema as a *directed arc* from R1.FK to R2.PK.

Referential integrity constraints displayed on the COMPANY relational database schema diagram.



4. Update Operations

- Three basic update operations on relations:
  Insert tuples, Delete tuples, Update (Modify) tuples

- Whenever update operations are applied, *the integrity constraints specified on the database schema should not be violated.*
- A DBMS that enforces the constraints *rejects* any operation that violates the constraints.
- It would be useful if the *system could explain to the user why the operation is rejected*.
- The user can *correct the reason for rejecting an operation* so that the operation can be performed.
- The system can be instructed to automatically correct the reason for rejecting a Delete or Update operation that violates referential integrity constraints.

4.1. The insert operation (1)

- The **insert operation** provides a list of attribute values for a new tuple that is to be inserted into a relation.

•Example:

R_1

A	B
a	b

R_2

C	D
c	a

dom(A) = alphabetic characters

Insert (a',b') into R_1

R_1

A	B
a	b
a'	b'

R_2

C	D
c	a

4.1. The insert operation (2)

- Insert can violate *any of the four types of constraints*.

- Example:

R_1

A	B
a	b

R_2

C	D
c	a

dom(A) = set of alphabetic characters
D foreign key of R_2 that refers to A

Insert (1, b) into R_1 violates the *domain constraint*

Insert (a, b') into R_1 violates the *key constraint*

Insert (null, b') into R_1 violates the *entity integrity constraint*

Insert (c', b') into R_2 violates the *referential integrity constraint*

4.1. The insert operation (3)

Actions to be taken to correct the reason for rejecting an insertion :

- Modify appropriately the tuple to be inserted.
- Modify the state of the database before the insertion operation.

4.1. The insert operation (4)

Example: **What constraints are violated? What actions can be taken to correct the reason for rejecting the insertion?**

Insert into EMPLOYEE

<‘Cecilia’, ‘F’, 123, ‘677678989’, null, ‘6357 Windy Lane, Katy, TX’, F, 28000, null, 4>

<‘Cecilia’, ‘F’, ‘Kolosky’, null, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, null, 4>

<‘Alicia’, ‘J’, ‘Zelaya’, ‘999887777’, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, ‘987654321’, 4>

<‘Cecilia’, ‘F’, ‘Kolosky’, ‘677678989’, ‘1960-04-05’, ‘6357 Windswept, Katy, TX’, F, 28000, ‘987654321’, 7>

<‘Cecilia’, ‘F’, ‘Kolosky’, ‘677678989’, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, ‘987654321’, 4>

4.1. The insert operation (5)

Example: **What constraints are violated? What actions can be taken to correct the reason for rejecting the insertion?**

Insert into EMPLOYEE

<‘Cecilia’, ‘F’, 123, null, null, ‘6357 Windy Lane, Katy, TX’, F, 28000, null, 7>

<‘Cecilia’, ‘F’, ‘Kolosky’, ‘999887777’, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, null, 7>

< ‘Cecilia’, ‘F’, ‘Kolosky’, ‘999888888’, ‘1960-04-05’, ‘6357 Windy Lane, Katy, TX’, F, 28000, ‘999999777’, 7 >

4.2. The delete operation (1)

- To specify **deletion**, a condition on the attributes of the relation selects the tuples to be deleted.
- Example:

R_1

A	B
a	b
a'	b'

R_2

C	D
1	a
2	a'

- **Delete the R_2 tuple(s) with C < 5**

R_1

A	B
a	b
a'	b'

R_2

C	D

- **Delete the R_1 tuple(s) with A = e'.**
  (no tuple qualifies)

4.2. The delete operation (2)

- Delete can violate *only the referential integrity constraint*.

- Example:

R_1	A	B
	a	b

R_2	C	D
	c	a

D is a foreign key of R_2 that refers to A

The operation:

Delete the tuple(s) of R_1 with B='b'

violates the *referential integrity constraint*. Why?

4.2. The delete operation (3)

Actions to be taken to correct the reason for rejecting a deletion:

- **Cascade (or propagate)** the deletion by deleting the tuples that reference the tuple that is being deleted.

  Note that this may cause the deletion of tuples in multiple relations.

- **Modify the referencing attributes values** that cause the violation.
  - Change these values to reference another valid tuple.
  - Set these values to null.
    Note that this is not possible if the referencing attribute is part of the primary key (entity integrity constraint violation).

- **Combinations of the last two options.**

4.2. The delete operation (4)

Example: **What constraints are violated? What actions can be taken to correct the reason for rejecting the deletion?**

1. Delete the tuple(s) of WORKS_ON with
ESSN=‘999887777’ and PNO = 10

2. Delete the tuple(s) of EMPLOYEE with
SSN=‘999000000’

3. Delete the tuple(s) of EMPLOYEE with
SSN=‘999887777’

4. Delete the tuple(s) of EMPLOYEE with
SSN=‘333445555’

4.2. The update operation (1)

- The **update operation** is used to change the values of one or more attributes in a tuple (or tuples) of a relation.

- It is necessary to specify a *condition* to select the tuples to be modified and *new values* for the attributes to be modified.

•Example:

R_1	A	B	C
	a	b	c
	a'	b	c'
	a''	b'	c'

Update attribute C of the tuples of R_1 with B=b to c".

R_1	A	B	C
	a	b	c''
	a'	b	c''
	a''	b'	c'

4.2. The update operation (2)

- Updating an attribute that is **neither the primary key nor a foreign key** attribute can violate only the **domain constraint**.

- Updating an attribute that is **part of the primary key** is similar to *deleting one tuple and inserting a new one*. Thus **all the constraints can be violated**.

- Updating a foreign key attribute can violate the **referential integrity constraint** and the **domain constraint**.

4.2. The update operation (3)

Example: **What constraints are violated?**

1. Update the SALARY of the EMPLOYEE tuple(s) with SSN=‘999887777’ to 28000

2. Update the DNO of the EMPLOYEE tuple(s) with SSN=‘999887777’ to 7

3. Update the DNO of the EMPLOYEE tuple(s) with SSN=‘999887777’ to 1

4. Update the SSN of the EMPLOYEE tuple(s) with SSN=‘999887777’ to ‘987654321’